

Solving NPuzzle with search algorithms

Pedro Fernandes (up201603846)
FEUP, MIEIC
Porto, Portugal
up201603846@fe.up.pt

Abstract— In this report it is described the implementation of the game NPuzzle. The game consists on a board of pieces that must be placed in a certain order. The language chosen for the implementation is Python. The main goal of the present work is to implement several search algorithms to solve the problem and compare their memory and time efficiency as well as understanding particular characteristics that make each algorithm more suitable for different problems.

Keywords—*Artificial Intelligence, Search Algorithms, Breadth First Search, Greedy Search, A*, Python*

I. INTRODUCTION

This work was developed for the course IART (Artificial Intelligence), as a resolution of the second online activity. On this report there will be a technical description of the game, but that doesn't involve any details about the implementation on chapter II - **Formulation of the problem**. On chapter III - **Experiences and Results** it is analysed the time and space efficiency. Several experiments were made on the course of this work and they will all be detailed on this chapter. On chapter IV - **Conclusions** it is summarized the final results of the project and drawn some final conclusions on it.

II. FORMULATION OF THE PROBLEM

The state is represented by a matrix of numbers representing each piece. This matrix should be 3x3 or 4x4, but theoretically it could have larger sizes, although it would be harder to solve.

The map configuration is loaded from a file and stored in the state. The goal state of the process is when one can read the matrix sequentially (left to right, and up to down), starting from 1 and ending at 0.

There are 4 possible operators, Up, Left, Right and Down:

- Switch 0 piece with the one above it
 - **pre-condition:** the 0 piece is not on the first row.
- Switch 0 piece with the one below it
 - **pre-condition:** the 0 piece is not on the last row.
- Switch 0 piece with the one to its left
 - **pre-condition:** the 0 piece is not on the first column.

- Switch 0 piece with the one to its right
 - **pre-condition:** the 0 piece is not on the last column.

The cost of every operator (move) is 1.

The final cost of the solution is the number of moves performed, meaning, is the sum of all move costs (which are 1). The goal is to solve the problem in the least number of moves.

III. EXPERIENCES AND RESULTS

The stats used to measure the performance of the algorithms were generated automatically by **stats.py** to an online google spreadsheet, which can be found [here](#). For each combination of level and algorithm, the program was executed 7 times. The highest and lowest time are disregarded and the middle 5 values are averaged and used for analysis. It totals out to $7(\text{tries}) \cdot 4(\text{algorithms}) \cdot 4(\text{levels}) = 112$ runs. Each run times out at 200 seconds. Below, there's tables comparing the results of running the algorithms with the given levels, with respects to number of moves, number of nodes explored and average time to solve. As specified, the first heuristic is the number of pieces out of place, and the second is the sum of the Manhattan distances of the pieces to their respective place.

	BFS	GS1	GS2	A*1	A*2
1	4	4	4	4	4
2	7	7	7	7	7
3	10	10	108	10	10
4	10	18	14	10	10

Table 1: Number of moves

Breadth first search and both A* seem to find the same, and best solutions, so it's hard to compare the heuristics in this case.. Greedy search does the same for the 3x3 levels, but seems to have very bad results on the 4x4 puzzles.

	BFS	GS1	GS2	A*1	A*2
1	24	12	4	4	5
2	108	15	7	7	9
3	584	15	1984	24	24
4	3152	265	58	19	65

Table 2: Number of nodes explored

Memory wise, A* once again proves to be the best algorithm, while breadth first search has a seemingly exponential growth. Greedy search has fairly good results, expect the second heuristic on the third level.

	BFS	GS1	GS2	A*1	A*2
1	0,011	0,008	0,004	0,002	0,005
2	0,050	0,010	0,005	0,005	0,006
3	0,288	0,011	2,846	0,015	0,015
4	10,366	0,192	0,042	0,016	0,045

Table 3: Average time to solve each level (in seconds)

Finally, all the algorithms show good to great results time wise. Only breadth first search takes more time on the last level, and as expected, A* has the very best results.

IV. CONCLUSIONS

This work allowed me to understand that if well formulated, any popular game can be implemented and solved with ease using search algorithms. The results are not very similar to those of the first project, but it's easy to understand why. In this game, the levels tested had much less complexity than those used in the previous one (Bloxorz). However, like verified in the previous section, the A* algorithm remains the most appropriate to use in these categories of games, with optimal moves and fantastic memory and time efficiency.

REFERENCES

- [1] aimacode, "aima-python", URL: <https://github.com/aimacode/aima-python>, visited on March 17th
- [2] Red Blob Games, "Implementation of A*", URL: <https://www.redblobgames.com/pathfinding/a-star/implementation.html>, visited on March 14th
- [3] Breadth-first search, URL: https://en.wikipedia.org/wiki/Breadth-first_search, visited on March 26th
- [4] Search algorithms part 2: Uniform Search Algorithms, URL: <https://medium.com/kredo-ai-engineering/search-algorithms-part-2-uniform-search-algorithms-1-be5583a2f1e1>, visited on March 27th